3.ATTENTION

06 February 2026 13:51

transduction → when a model learns to predict data instead of learning a general rule that works for all future data

induction → learn a general function → Apply it to new data

transductive learning → the model uses the training data and the test inputs during learning but doesn't learn any general rule

for RNN'S/LSTM'S/GRU'S

$$h_t = f(h_{t-1}, x_t)$$

at each time step output memory → input + previous memory

? why this prevents parallelization

i) to compute h_t you must have h_{t-1}, you cannot compute everything at once (sequential bottleneck)

ii) during training you store all hidden states for back-propagation $h_1 h_2 \ldots h_t$ → for longer sequences → more steps
→ more stored activations
→ more GPU memory

so you need to reduce batch size which makes training slower

Extended Neural GPU, ByteNet, ConvS2S tried solving the sequential bottleneck problem

all of these use CNN's → computing hidden representations in parallel for all inputs & output positions

A convolutional layer applies filter to every position simultaneously

the no of operations required to relate signals from two arbitrary input or output positions grows with distance between points (linearly → ConvS2S
logarithmically → ByteNet)

In transformers, we get around this with constant number of operations → reduced effective resolution due to averaging attention weighted positions → we can counteract with Multi-head Attention

self Attention an attention mechanism relating different positions of a single sequence in order to compute the representation of the sequence

Model Architecture -

Encoder - Decoder Structure

maps an input sequence of symbol representations $(x_1, x_2, x_3 \ldots x_n)$ to a sequence of continuous representations $z = (z_1, z_2 \ldots z_n)$

Generates an output sequence $(y_1, y_2 \ldots y_m)$ of symbols one element at a time

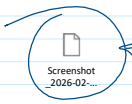
Model → Auto regressive at each step

(consumes previously generated symbols as additional inputs when generating the text

i) Attention → tokens talk to each other

ii) Point wise FFN → each token processes itself

point wise fully connected layers doesn't mean small, it means applied independently at each position



← Transformer Model Architecture

Encoder block · stack of N=6 identical layers

Each layer has two sub-layers → Multi-head Self Attention Mechanism
→ simple position wise fully connected feed forward network

A residual connection around each of two sublayers, followed by layer normalization

↓

Output of each sublayer → LayerNorm $(x + \text{sublayer}(x))$

↳ function implemented by sublayer

→ to facilitate these residual connections

↳ all sublayers as well as embedding layers produce outputs of dimension $d_{model} = 512$

Decoder block -

Also composed of stack of N=6 identical layers

In addition to the two sublayers of encoder, there is a third sublayer

third sublayer → performs multi head attention over the output of encoder stack

All layers have residual connections followed by layer normalization

* in the decoder block

↳ we modify the self attention sublayer to prevent attending to subsequent positions

↓

This masking, along with output embeddings offset by one position ensure predictions for position 'i' can depend only on known outputs at positions less than i

Attention function → mapping a set of Query & key-value pairs to an output

Attention function → mapping a set of Query & key-value pairs to an output
All are vectors

Output → weighted sum of values
weight assigned to each value is computed by a compatibility function of the query with corresponding key

Scaled Dot product Attention -

d_k → dimension of keys
d_v → dimension of values

Screenshot 2026-02-...

→ scaled dot product Attention
&
MHA with several attention layers in parallel

dot products of Query with all keys, divide by $\sqrt{d_k}$ & apply a softmax function to obtain the weights of the values

→ In practice
↳ set of queries simultaneously packed together into a matrix Q, keys & values packed together into K & V

$$\text{Attention}(Q,K,V) = \text{Softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

→ $\text{score}(q,k) = q^T k$ while scaling in transformer prevents huge values

two most commonly used attention functions are additive attention & dot product (multiplicative attention)
→ very fast, fewer parameters
less expressive than additive in theory

(Bahdanau attention 2014)

$\text{score}(q,k) = v^T \tanh(W_q q + W_k k)$

instead of simply multiplying matrices →
1) transform q & k with learned matrices
2) add them
3) pass them through tanh
4) project with vector v

(score is computed with a small neural network)

↓
expressive, learns complex interactions
slower, more parameters
} powerful but heavier

dot product attention is similar except for scaling factor ($1/\sqrt{d_k}$)
Additive attention computes the compatibility function using a FFN with single layer
→ both are similar in theoretical complexity, but dot product is fast & space efficient

for small values of d_k, two mechanisms perform similarly
for large values additive attention outperforms dot product without scaling
→ Assumption that dot products grow in magnitude pushing softmax function to have extremely low gradients
To counteract this, we scale the dot products by $1/\sqrt{d_k}$

Multi-head Attention -

instead of performing a single attention function → it is beneficial to linearly project q, k, & v h times with different, learned linear projections to d_q, d_k & d_v dimensions

On each of these projected versions of q, k and v we perform the attention fn in parallel yielding d_v dimensional output values
These are concatenated & projected once again, resulting in the final values as depicted in figure above from earlier

MHA allows the model to attend to information from different representation subspaces at different positions

$$\text{MultiHead}(Q,K,V) = \text{Concat}(\text{head}_1 \dots \text{head}_h)W^O$$
$$\text{where head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

projections are the parameter matrices
$W_i^Q \in \mathbb{R}^{d_{model} \times d_k}$
$W_i^K \in \mathbb{R}^{d_{model} \times d_k}$
$W_i^V \in \mathbb{R}^{d_{model} \times d_v}$
$W^O \in \mathbb{R}^{hd_v \times d_{model}}$

in the paper they employed $h=8$
for each head $d_k = d_v = d_{model}/h = 64$
} the reduced dimension of each head
total computation cost is similar to that of single head attention with full dimensionality

Transformer uses MHA in 3 ways -

→ In encoder-decoder attention layers, the queries come from previous decoder layer, the memory keys & values from output of encoder layer
This allows every position in decoder to attend over all positions in the input sequence ⇒ it mirrors typical seq2seq model's encoder decoder structure

→ The encoder contains self attention layers, here all q, k, v come from same place i.e output of previous layer in the encoder
each position of encoder can attend to all positions in the previous layer of the encoder

→ similarly, self attention in decoder allows each position to attend to all positions in the decoder up to and including that position
To preserve the autoregressive property, we try to prevent leftward information flow in the decoder
↳ This is implemented by masking out (setting to $-\infty$) all values in the input of the softmax which correspond to illegal connections

} Auto regression here implies model generating tokens only using previously generated tokens
$P(y_1, y_2 \dots y_T) = \prod_{i=1}^{T} (P_{y_i} | y_1 \dots y_{i-1})$

Position wise Feed forward Networks

in addition to attention layers, each of the layers in encoder & decoder contains a fully connected FFN, it is applied to each position separately & identically

Position wise Feed forward Networks

in addition to attention layers, each of the layers in encoder & decoder contains a fully connected FFN, it is applied to each position separately & identically

This consists of two linear transformations with a ReLu in b/w

$$FFN(x) = \max(0, xW_1 + b_1)W_2 + b_2$$

while the transformations are same across positions they use different parameters from layer to layer

The dimensionality of input & output is $d_{model} = 512$ The inner layer dim $= d_{ff} = 2048$

Embedding & Softmax.

→ uses learned embeddings to convert input & output tokens to vectors of dimension d_{model}

→ we use the learned linear transformations & softmax to convert decoder output to predict next token probabilities

→ we share same weight matrix between the two embedding layers & the pre softmax linear transformation → we multiply those wts by $\sqrt{d_{model}}$

- say that the geometry used for representing words is the same for classifying them
- encoder & decoder input normally $E, \text{encoder}, E \in R^{n}(V \times d_{model})$ → embedding dimension, Vocab size
- reduces parameters, improves generalization
- embeddings are scaled
- $x = \sqrt{d_{model}}$ embedding
- This stabilizes variance so embeddings dont become tiny relative to positional encodings & attention activations

they use the same matrix here too

Normally logits $= W_o h \rightarrow \mathbb{R}(V \times d_{model})$

here $W_o = E^T$

output classifier matrix = Embedding matrix

WEIGHT TYING

Positional Encoding

↳ we dont have recurrence/convolution to make use of sequence order

↳ for this, to inject some info about position of tokens, we add positional encodings to the input embeddings at the bottom of encoder & decoder stacks

Screenshot 2026-02-...

refer to this table for comparisons

positional encodings dimension → d_{model}

in this paper they use sin & cosine fn of diff frequencies

$$PE(pos, 2i) = \sin\left(pos / 10000^{2i/d_{model}}\right)$$

$$PE(pos, 2i+1) = \cos\left(pos / 10000^{2i/d_{model}}\right)$$

pos → position, i → dimension

self attention complexity $O(n^2 d)$ n → seq length, d → representation dimension

each token attends to every other token $n \times n$ interactions

parallelizable so sequential operations $O(1)$

Any token talks to any other in one step Max path length $O(1)$

Recurrent Networks

Complexity $O(n d^2)$ Must process sequentially

Sequential ops $O(n)$

path length $O(n)$ → information must travel step by step

slow + long dependencies are hard

RNN → walk

CNN → Hop

Transformers → Teleport

Convolutional networks

Complexity per layer $O(k n d^2)$ → kernel size of convolution, parallel across positions

But distant tokens need multiple layers

Max path length $= O(\log_k(n))$

Restricted attention

Complexity per layer $O(r n d)$ → local attention window

Max path length $O(n/r)$ path length increases

trade off b/w efficiency & reach

{Transformers win → Max path length $O(1)$ → no propagation delay → Any token can influence any other immediately → long range dependencies are easier}

in depth PE

→ What problem? → Assign coordinates to positions

it is about building continuous coordinate system

- token 0 → vector → should be unique
- token 1 → vector → encode order
- ⋮ → allow relative short reasoning, work for long sequences, be smooth & not arbitrary IDs

Using integers → ✗ Bad idea → scale explodes / no geometric relationships / hard to generalize / scale explodes

Instead of numbers imagine describing a location using waves

each wave oscillates, the combination uniquely identifies position, each dimension is a wave is what sinusoidal does

The sine cosine pair together form a circle → sine alone loses phase information, sine + cosine gives full angle

This lets model represent position as rotation in space {Position becomes an angle}

some waves change fast (good locally)
some change slow (good for globally)
} together they uniquely identify any position

Here a shift in position becomes a linear transformation encoding(pos+k) can be computed from encoding(pos) using matrix multiplication

Having linear relationships makes it easy to learn relative positions

10000 just spreads frequencies exponentially ensuring short range precision & long range stability

Having linear relationships makes it easy to learn relative positions

10000 just spreads frequencies exponentially ensuring short range precision & long range stability

Basically a token gets coordinates like (pos on footwave, pos on medwave,) model learns to read these as geometry & not numbers

Sinusoidal position Encoding turns token positions into a multi frequency geometric coordinate system where relative shifts become linear operations

training results & pointers

Transformer (big) achieved 28.4 BLEU on WMT 2014 eng-german

Transformer (base) 27.3 BLEU . 3.5 days on 8 P100 GPUs

Better than CNN/RNN seq2seq models
Much cheaper to train

Self attention beats recurrence + convolution for machine translation

Model	layers	d_model	Heads	Parameters	BLEU
Base	6	512	8	~65M	27.3
big	6	1024	16	~213M	28.4

Bigger model = wider model → large hidden dimension
more heads
more dropout

They used Adam optimizer

learning rate schedule points

1. warmup phase → LR increases linearly
2. Decay phase → LR decays proportional to $1/\sqrt{step}$

Regularization tricks

1. Dropout (0.1 for base)
   Applied to sublayer outputs
   attention
   embeddings
2. Residual connections
   Every sublayer uses
   LayerNorm (x + sublayer(x))
   to stabilize deep training
3. $\varepsilon_{ls} = 0.1$
   → slightly worse perplexity
   → Better BLEU/accuracy
   (because the model doesn't become over confident)

Ablation insights

1. No attention heads
   too few hurts performance
   MHA learns diverse representations
2. Model dimension matters
   d_model, d_ff > improve quality / increases compute
3. Dropout helps generalization
   Higher dropout (0.2 - 0.3)
   prevents overfitting
   especially important for bigger models
4. Positional Encoding
   Sinusoidal v learned
   Almost identical performance

Transformers cost is dominated by
Attention $O(n^2)$
but benefits
→ No recurrence
→ parallelizable
→ faster training

positions signal is necessary
exact form is not critical
less critical